

从 0 开始 Python 快速入门：面向深度学习实践

0. 这份手册学什么

本手册重点讲后面会马上用到的内容：

1. 软件安装：Miniconda、VS Code、课程环境。
2. Python 运行方式：终端运行、脚本运行、VS Code 运行。
3. 基础语法：变量、数据类型、列表、字典、条件判断、循环。
4. 函数：如何把重复代码封装起来。
5. 模块与包：`import`、`pip`、`conda` 是什么。
6. 文件路径与读写：后续保存模型、日志、图片会用到。
7. NumPy 基础：深度学习中理解数组、矩阵、`shape` 的前置知识。
8. Matplotlib 基础：查看图片、曲线和实验结果。
9. 类与对象：理解 PyTorch 中 `nn.Module` 的前置知识。
10. PyTorch 前置概念：`tensor`、`device`、`batch`、`shape`。
11. 调试方法：怎么看报错、怎么打印 `shape`、怎么定位问题。

1. 学习路径总览

建议按下面顺序学习：

安装工具

- > 创建课程环境
- > 会运行 Python 脚本
- > 掌握变量、列表、字典
- > 掌握 `if` / `for` / 函数
- > 会使用 `import` 和 `pip`
- > 会处理路径和文件
- > 会看懂 NumPy 数组 `shape`
- > 会看懂 PyTorch `tensor` 和 `device`
- > 能读懂 MNIST 训练脚本

学习 Python 时不要追求“一次全记住”。更重要的是：

1. 会运行代码。
2. 看得懂报错。
3. 知道该打印什么来检查。
4. 能小步修改并观察结果。

2. 需要安装哪些软件

2.1 推荐软件清单

软件	作用	是否必须
Miniconda	管理 Python 和课程环境	必须
VS Code	写代码、运行脚本、查看文件	推荐
Python 扩展	让 VS Code 支持 Python 解释器、调试、代码提示	推荐
终端	输入命令、运行脚本	必须

本课程建议使用 Miniconda，而不是直接安装系统 Python。原因是后续实验会安装 PyTorch、torchvision、matplotlib、tqdm 等包，如果都装进系统 Python，后面很容易混乱。

2.2 Miniconda、Anaconda、Python 的区别

名称	可以怎么理解
Python	编程语言本身，也是运行 <code>.py</code> 文件的解释器
Conda	环境和包管理工具，可以创建多个隔离环境
Miniconda	只包含 conda、Python 和少量基础包，轻量，适合课程
Anaconda	包含大量数据科学包，体积大，安装慢

本课程推荐 Miniconda。

3. 安装 Miniconda

3.1 下载入口

windows: https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-py311_26.1.1-1-Windows-x86_64.exe

macos: https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-py311_26.1.1-1-MacOSX-arm64.pkg

3.2 Windows 安装步骤

1. 双击 Miniconda 安装包。

2. 安装范围建议选择 `Just Me`。
3. 安装路径建议使用英文路径，例如：

```
C:\Users\你的用户名\miniconda3
```

不要安装到中文路径或带空格的路径里。

4. 安装选项建议：
 - 勾选 `Add Miniconda3 to my PATH environment variable`。
 - 如果你没有其他 Python 需求，可以勾选 `Register Miniconda3 as my default Python`。

5. 安装完成后，从开始菜单打开 `Anaconda Prompt`。

6. 输入：

```
conda --version
```

如果看到类似：

```
conda x.x.x
```

说明安装成功。

3.3 macOS 安装步骤

1. 下载对应芯片的 `.pkg` 安装包。
2. 双击安装。
3. 打开 Terminal。
4. 输入：

```
conda --version
```

如果提示找不到 `conda`，尝试：

```
~/miniconda3/bin/conda init zsh
```

关闭 Terminal，重新打开。

如果你使用 `bash`：

```
~/miniconda3/bin/conda init bash
```

4. 安装 VS Code 与 Python 扩展

4.1 下载 VS Code

官方页面：

<https://code.visualstudio.com/>

安装时保持默认选项即可。Windows 用户建议勾选：

- Add to PATH
- Open with Code

这些选项会让后续打开文件夹更方便。

4.2 安装 Python 扩展

打开 VS Code 后：

1. 点击左侧 Extensions。
2. 搜索 `Python`。
3. 安装 Microsoft 发布的 Python 扩展。

官方说明：

<https://code.visualstudio.com/docs/languages/python>

5. 创建课程 Python 环境

5.1 为什么要创建环境

一个电脑上可以有多个 Python 环境：

```
base
ei-python
ei-mnist
ei-detection
ei-vla
```

每个环境可以安装不同版本的包。这样做的好处是：一个实验坏了，不会影响另一个实验。

5.2 创建本手册环境

打开 Anaconda Prompt 或 Terminal：

```
pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

```
conda config --set show_channel_urls yes
```

在不同系统下的 `.condarc` 目录如下：

- Linux: `${HOME}/.condarc`
- macOS: `${HOME}/.condarc`
- Windows: `C:\Users\<YourUserName>\.condarc`

修改文件为：

```
channels:
  - defaults
show_channel_urls: true
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
```

执行：

```
conda create -n py_learning python=3.10 -y
```

激活环境：

```
conda activate py_learning
```

如果终端开头出现：

```
(py_learning)
```

说明你已经进入课程环境。

5.3 检查 Python 版本

```
python --version
```

期望看到：

```
Python 3.10.x
```

5.4 安装基础包

```
pip install numpy matplotlib tqdm
```

5.6 常用 conda 命令

命令	作用
<code>conda env list</code>	查看所有环境
<code>conda activate ei-python</code>	进入环境
<code>conda deactivate</code>	退出当前环境
<code>conda env remove -n ei-python</code>	删除环境
<code>python -m pip list</code>	查看当前环境安装了哪些 Python 包
<code>python -m pip show numpy</code>	查看某个包的信息

6. 创建课程文件夹与第一个 Python 文件

6.1 创建文件夹

Windows:

```
mkdir D:\embodied_ai_python
cd D:\embodied_ai_python
```

macOS/Linux:

```
mkdir -p ~/embodied_ai_python
cd ~/embodied_ai_python
```

推荐目录结构:

```
embodied_ai_python/
  01_hello.py
  02_variables.py
  03_list_dict.py
  04_numpy_intro.py
  outputs/
```

6.2 选择 Python 解释器

打开课程文件夹后:

1. 按 `Ctrl + Shift + P`, macOS 为 `Command + Shift + P`。
2. 输入 `Python: Select Interpreter`。
3. 选择包含课程环境名称的解释器, 例如 `ei-python`。

如果选错解释器, 常见表现是:

```
ModuleNotFoundError: No module named 'torch'
```

这不一定是没安装，可能是 VS Code 用了另一个 Python。

6.2 第一个 Python 文件

新建 `01_hello.py`：

```
print("Hello, embodied intelligence!")
print("这是我的第一个 Python 程序。")
```

运行：

```
python 01_hello.py
```

输出：

```
Hello, embodied intelligence!
这是我的第一个 Python 程序。
```

Note

无法加载文件 D:\Documents\WindowsPowerShell\profile.ps1，因为在此系统上禁止运行脚本。有关详细信息，请参阅

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

6.3 Python 脚本如何执行

当你运行：

```
python 01_hello.py
```

含义是：

1. 调用当前环境里的 Python 解释器。
2. 让解释器读取 `01_hello.py`。
3. 从上到下一行一行执行。

Python 对缩进非常敏感。后面看到 `if`、`for`、`def`、`class` 时，缩进就是代码结构的一部分。

7. 变量与基本数据类型

7.1 变量是什么

变量可以理解为“给一个值起名字”。

```
course_name = "具身智能感知基础"
epoch = 2
learning_rate = 0.001
use_gpu = True

print(course_name)
print(epoch)
print(learning_rate)
print(use_gpu)
```

解释：

变量	类型	含义
course_name	字符串 <code>str</code>	课程名称
epoch	整数 <code>int</code>	训练轮数
learning_rate	浮点数 <code>float</code>	学习率
use_gpu	布尔值 <code>bool</code>	是否使用 GPU

7.2 查看类型

```
x = 10
y = 3.14
name = "MNIST"
ok = True

print(type(x))
print(type(y))
print(type(name))
print(type(ok))
```

输出会类似：

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

深度学习代码里经常会看到：

```
batch_size = 64
num_classes = 10
lr = 1e-3
```

其中 `1e-3` 等于 `0.001`。

7.3 字符串与 f-string

字符串用来表示文本：

```
model_name = "SmallCNN"
accuracy = 0.9823

message = f"模型 {model_name} 的准确率是 {accuracy:.2%}"
print(message)
```

输出：

```
模型 SmallCNN 的准确率是 98.23%
```

深度学习训练日志经常用 f-string：

```
epoch = 1
loss = 0.2381
acc = 0.9456

print(f"Epoch {epoch}: loss={loss:.4f}, acc={acc:.4f}")
```

输出：

```
Epoch 1: loss=0.2381, acc=0.9456
```

7.4 常见运算

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

含义：

运算	含义
<code>+</code>	加
<code>-</code>	减
<code>*</code>	乘
<code>/</code>	除
<code>//</code>	整除
<code>%</code>	取余
<code>**</code>	幂

8. 列表、元组和字典

8.1 列表 list

列表用来保存一组数据。

```
labels = [0, 1, 2, 3, 4]
print(labels)
print(labels[0])
print(labels[1])
print(labels[-1])
```

注意：Python 下标从 0 开始。

```
labels[0] 是第 1 个元素
labels[1] 是第 2 个元素
labels[-1] 是最后一个元素
```

8.2 列表切片

```
numbers = [0, 1, 2, 3, 4, 5, 6]

print(numbers[0:3])
print(numbers[2:5])
print(numbers[:4])
print(numbers[4:])
```

解释：

写法	含义
<code>numbers[0:3]</code>	取下标 0、1、2
<code>numbers[:4]</code>	从开头取到下标 3
<code>numbers[4:]</code>	从下标 4 取到最后

切片规则是“左闭右开”：包含左边，不包含右边。

8.3 列表追加与长度

```
losses = []

losses.append(0.8)
losses.append(0.4)
losses.append(0.2)

print(losses)
print(len(losses))
```

训练时常见写法：

```
history = []
history.append({"epoch": 1, "loss": 0.8, "acc": 0.75})
history.append({"epoch": 2, "loss": 0.4, "acc": 0.88})

print(history)
```

8.4 元组 tuple

元组和列表类似，但通常表示固定结构。

```
image_shape = (1, 28, 28)
print(image_shape)
print(image_shape[0])
```

在深度学习中，shape 通常会用元组表示：

```
(batch_size, channel, height, width)
```

例如：

```
(64, 1, 28, 28)
```

8.5 字典 dict

字典用 key-value 保存数据。

```
config = {
    "batch_size": 64,
    "learning_rate": 0.001,
    "epochs": 2,
    "model": "SmallCNN",
}

print(config["batch_size"])
print(config["model"])
```

字典非常适合保存实验配置。

```
result = {
    "train_loss": 0.12,
    "train_acc": 0.96,
    "test_acc": 0.95,
}

print(f"测试准确率: {result['test_acc']:.2%}")
```

9. 条件判断

9.1 if 语句

```
accuracy = 0.93

if accuracy >= 0.9:
    print("模型效果较好")
else:
    print("模型还需要改进")
```

注意缩进。if 后面的代码块必须缩进。

9.2 多分支判断

```
device = "cuda"

if device == "cuda":
    print("使用 NVIDIA GPU")
elif device == "mps":
    print("使用 macOS MPS")
else:
    print("使用 CPU")
```

后续 PyTorch 代码中会看到类似逻辑：

```
import torch

if torch.cuda.is_available():
    device = "cuda"
else:
    device = "cpu"

print(device)
```

9.3 比较运算

写法	含义
<code>a == b</code>	是否相等
<code>a != b</code>	是否不相等
<code>a > b</code>	大于
<code>a >= b</code>	大于等于
<code>a < b</code>	小于
<code>a <= b</code>	小于等于

注意：判断相等用 `==`，赋值用 `=`。

10. 循环

10.1 for 循环

```
for i in range(5):
    print(i)
```

输出：

```
0
1
2
3
4
```

`range(5)` 表示从 0 到 4。

训练代码中常见：

```
epochs = 3

for epoch in range(1, epochs + 1):
    print(f"开始第 {epoch} 轮训练")
```

输出：

```
开始第 1 轮训练
开始第 2 轮训练
开始第 3 轮训练
```

10.2 遍历列表

```
losses = [0.9, 0.5, 0.2]

for loss in losses:
    print(loss)
```

10.3 enumerate

如果既想要下标，又想要元素：

```
class_names = ["zero", "one", "two"]

for index, name in enumerate(class_names):
    print(index, name)
```

输出：

```
0 zero
1 one
2 two
```

10.4 zip

如果要同时遍历两个列表：

```
predictions = [0, 2, 1]
labels = [0, 1, 1]

for pred, label in zip(predictions, labels):
    print(f"预测={pred}, 真实={label}")
```

10.5 break 和 continue

```
for i in range(10):
    if i == 3:
        continue
    if i == 7:
        break
    print(i)
```

解释：

- `continue`：跳过本轮循环，进入下一轮。
- `break`：直接结束循环。

11. 函数

11.1 为什么需要函数

函数用来把一段代码封装起来，方便重复使用。

不使用函数：

```
loss1 = 0.4
acc1 = 0.9
print(f"loss={loss1:.4f}, acc={acc1:.4f}")

loss2 = 0.3
acc2 = 0.92
print(f"loss={loss2:.4f}, acc={acc2:.4f}")
```

使用函数：

```
def print_metrics(loss, acc):
    print(f"loss={loss:.4f}, acc={acc:.4f}")

print_metrics(0.4, 0.9)
print_metrics(0.3, 0.92)
```

11.2 函数的输入和输出

```
def add(a, b):
    result = a + b
    return result

x = add(3, 5)
print(x)
```

输出：

```
8
```

11.3 默认参数

```
def train_message(epoch, total_epochs=10):  
    print(f"Epoch {epoch}/{total_epochs}")  
  
train_message(1)  
train_message(1, total_epochs=3)
```

输出：

```
Epoch 1/10  
Epoch 1/3
```

11.4 深度学习中的函数风格

后续会看到类似函数：

```
def accuracy(predicted, labels):  
    correct = 0  
    total = len(labels)  
  
    for pred, label in zip(predicted, labels):  
        if pred == label:  
            correct += 1  
  
    return correct / total  
  
predicted = [0, 2, 1, 3]  
labels = [0, 1, 1, 3]  
acc = accuracy(predicted, labels)  
print(acc)
```

输出：

```
0.75
```

这个函数表达的就是“预测对了多少个 / 总共有多少个”。

12. 模块、包和 import

12.1 什么是 import

Python 有很多功能不是内置的，需要导入模块。

```
import math

print(math.sqrt(16))
print(math.pi)
```

12.2 常见导入方式

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

解释：

写法	含义
<code>import numpy as np</code>	导入 numpy，并起别名 np
<code>import matplotlib.pyplot as plt</code>	导入画图模块，并起别名 plt
<code>from pathlib import Path</code>	从 pathlib 中导入 Path

为什么常用别名？

```
import numpy as np
```

比每次写：

```
import numpy
```

然后使用 `numpy.array(...)` 更简洁。

12.3 包从哪里来

当你运行：

```
python -m pip install numpy
```

意思是：在当前 Python 环境中安装 `numpy` 包。

安装后就可以：

```
import numpy as np
```

如果报错：

```
ModuleNotFoundError: No module named 'numpy'
```

通常说明当前环境没有安装这个包，或者 VS Code 选错了解释器。

13. 文件路径与文件读写

13.1 为什么要学路径

深度学习实验会频繁读写文件：

文件	例子
数据集	data/MNIST
训练日志	runs/training_log.csv
图片结果	runs/predictions.png
模型权重	saved_models/model.pt

如果路径写错，程序找不到文件。

13.2 使用 pathlib

推荐使用 `pathlib.Path`。

```
from pathlib import Path

project_dir = Path(".")
output_dir = project_dir / "outputs"
output_dir.mkdir(parents=True, exist_ok=True)

print(output_dir)
```

解释：

- `Path(".")` 表示当前目录。
- `/` 可以拼接路径。
- `mkdir` 创建文件夹。
- `parents=True` 表示父目录不存在时一起创建。
- `exist_ok=True` 表示目录已存在也不报错。

13.3 写文本文件

```
from pathlib import Path

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)

log_path = output_dir / "log.txt"
log_path.write_text("hello python\n", encoding="utf-8")

print("写入完成: ", log_path)
```

13.4 读文本文件

```
from pathlib import Path

log_path = Path("outputs") / "log.txt"
content = log_path.read_text(encoding="utf-8")

print(content)
```

13.5 写 CSV 风格日志

```
from pathlib import Path

rows = [
    {"epoch": 1, "loss": 0.8, "acc": 0.75},
    {"epoch": 2, "loss": 0.4, "acc": 0.88},
]

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)
csv_path = output_dir / "training_log.csv"

with csv_path.open("w", encoding="utf-8") as f:
    f.write("epoch,loss,acc\n")
    for row in rows:
        f.write(f"{row['epoch']},{row['loss']},{row['acc']}\n")

print("保存到: ", csv_path)
```

后续 MNIST 实验会保存类似的训练日志。

14. NumPy 入门：理解数组和 shape

14.1 为什么要学 NumPy

NumPy 是 Python 科学计算基础。深度学习中的 tensor 与 NumPy 数组非常相似，都强调：

1. 多维数组。

2. shape。
3. 矩阵运算。
4. 批量计算。

14.2 创建数组

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[1, 2, 3], [4, 5, 6]])

print(a)
print(b)
print(a.shape)
print(b.shape)
```

输出：

```
[1 2 3]
[[1 2 3]
 [4 5 6]]
(3,)
(2, 3)
```

`b.shape == (2, 3)` 表示 2 行 3 列。

14.3 常用数组

```
import numpy as np

zeros = np.zeros((2, 3))
ones = np.ones((2, 3))
random_values = np.random.randn(2, 3)

print(zeros)
print(ones)
print(random_values)
```

14.4 数组运算

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([10, 20, 30])

print(x + y)
print(x * y)
print(x * 2)
```

NumPy 会对整个数组批量运算，不需要自己写循环。

14.5 二维数组切片

```
import numpy as np

image = np.arange(16).reshape(4, 4)
print(image)
print(image[0, 0])
print(image[0, :])
print(image[:, 0])
print(image[1:3, 1:3])
```

这和图像处理非常相关。灰度图可以理解为二维数组，彩色图可以理解为三维数组。

14.6 reshape

```
import numpy as np

x = np.arange(12)
print(x.shape)

y = x.reshape(3, 4)
print(y)
print(y.shape)
```

深度学习中经常需要调整 shape。比如 MNIST 图片是 `28 x 28`，一张图有 784 个像素。

```
import numpy as np

image = np.zeros((28, 28))
flat = image.reshape(28 * 28)

print(image.shape)
print(flat.shape)
```

输出：

```
(28, 28)
(784,)
```

14.7 axis 的概念

```
import numpy as np

x = np.array([[1, 2, 3], [4, 5, 6]])

print(np.sum(x))
print(np.sum(x, axis=0))
print(np.sum(x, axis=1))
```

解释：

- 不写 `axis`：所有元素求和。
- `axis=0`：沿着行方向压缩，得到每一列的和。
- `axis=1`：沿着列方向压缩，得到每一行的和。

深度学习中 `argmax(dim=1)`、`mean(dim=0)` 与这个思想类似。

15. Matplotlib 入门：画图和显示图片

15.1 画一条曲线

```
from pathlib import Path

import matplotlib.pyplot as plt

epochs = [1, 2, 3, 4]
losses = [0.9, 0.6, 0.4, 0.25]

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)

plt.figure()
plt.plot(epochs, losses, marker="o")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.title("Training Loss")
plt.savefig(output_dir / "loss_curve.png", dpi=150)
plt.close()
```

运行后会在 `outputs` 文件夹生成 `loss_curve.png`。

15.2 显示一张灰度图

```

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

image = np.zeros((28, 28))
image[8:20, 12:16] = 1.0

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)

plt.figure()
plt.imshow(image, cmap="gray")
plt.axis("off")
plt.savefig(output_dir / "fake_digit.png", dpi=150)
plt.close()

```

这段代码人为造了一张 `28 x 28` 的灰度图。MNIST 图片本质上也是类似的二维数组。

16. 类与对象：理解 PyTorch nn.Module 的前置知识

16.1 为什么要学 class

PyTorch 模型通常写成：

```

class SmallCNN(nn.Module):
    ...

```

所以必须知道类是什么、`self` 是什么、`__init__` 是什么。

16.2 一个最小类

```

class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def print_info(self):
        print(f"{self.name} 的分数是 {self.score}")

student = Student("小明", 95)
student.print_info()

```

解释：

写法	含义
<code>class Student</code>	定义一个类
<code>__init__</code>	创建对象时自动调用
<code>self.name</code>	对象自己的属性
<code>print_info</code>	对象的方法
<code>Student("小明", 95)</code>	创建一个对象

16.3 类保存状态

```
class AverageMeter:
    def __init__(self):
        self.total = 0.0
        self.count = 0

    def update(self, value):
        self.total += value
        self.count += 1

    def average(self):
        return self.total / self.count

meter = AverageMeter()
meter.update(0.8)
meter.update(0.4)
meter.update(0.2)

print(meter.average())
```

训练时经常需要累计 loss 和 accuracy，这种写法就很常见。

16.4 类和 PyTorch 模型的关系

先看一个普通类：


```
class LinearModel:
    def __init__(self, weight, bias):
        self.weight = weight
        self.bias = bias

    def forward(self, x):
        return self.weight * x + self.bias

model = LinearModel(weight=2, bias=1)
print(model.forward(3))
```

输出：

7

PyTorch 模型也是类似思想：模型内部保存参数，`forward` 定义输入如何变成输出。

17. PyTorch 前置概念：tensor、shape、device

本节只是为后续做铺垫，不要求一次完全掌握。

17.1 安装 PyTorch

本手册重点是 Python。PyTorch 的 CPU/GPU 安装会在实践手册中详细讲。若你已经要提前安装，可优先查看官方页面：

<https://docs.pytorch.org/get-started/locally/>

17.2 tensor 是什么

Tensor 可以理解为 PyTorch 中的多维数组。

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([10.0, 20.0, 30.0])

print(x + y)
print(x * y)
print(x.shape)
```

17.3 图像 tensor 的 shape

深度学习里图像常见 shape：

[batch_size, channels, height, width]

MNIST 示例：

```
[64, 1, 28, 28]
```

含义：

维度	含义
64	一次送入 64 张图片
1	灰度图，1 个通道
28	高度
28	宽度

17.4 device 是什么

device 表示数据和模型放在哪里计算：

```
import torch

if torch.cuda.is_available():
    device = torch.device("cuda")
else:
    device = torch.device("cpu")

x = torch.randn(2, 3).to(device)
print(x.device)
```

重要规则：模型和数据必须在同一个 device 上，否则会报错。

17.5 argmax

分类模型输出通常是一组分数。

```
import torch

scores = torch.tensor([[0.1, 2.5, 0.3], [3.0, 0.2, 0.1]])
predicted = scores.argmax(dim=1)

print(predicted)
```

输出：

```
tensor([1, 0])
```

解释：

- 第一行最大值在下标 1，所以预测类别是 1。
- 第二行最大值在下标 0，所以预测类别是 0。

MNIST 有 10 个类别，输出通常是 `[batch_size, 10]`。

18. 调试：怎么读报错，怎么检查 shape

18.1 报错不是坏事

初学者看到红色报错容易紧张。实际上，报错是程序告诉你哪里不对。

看报错时重点看最后几行：

```
ModuleNotFoundError: No module named 'numpy'
```

这说明没有安装 numpy，或当前环境选错。

18.2 常见报错

报错	常见原因	处理
<code>ModuleNotFoundError</code>	没装包或环境选错	<code>python -m pip install 包名</code> ，检查解释器
<code>FileNotFoundError</code>	路径写错	打印路径，确认文件存在
<code>SyntaxError</code>	语法错误	检查括号、冒号、引号
<code>IndentationError</code>	缩进错误	统一使用 4 个空格
<code>RuntimeError: Expected all tensors to be on the same device</code>	模型和数据不在同一设备	检查 <code>.to(device)</code>

18.3 多打印 shape

深度学习调试常用：

```
import torch

images = torch.randn(64, 1, 28, 28)
labels = torch.randint(0, 10, (64,))

print("images shape:", images.shape)
print("labels shape:", labels.shape)
```

如果 shape 不符合预期，就先别继续训练。

18.4 小步运行

不要一口气写 100 行再运行。建议：

1. 写 5-10 行。
2. 运行一次。
3. 打印关键变量。
4. 确认没问题再继续。

19. 面向深度学习的 Python 练习

19.1 练习 A：计算准确率

要求：补全代码，计算预测正确的比例。

```
predicted = [0, 2, 1, 3, 4]
labels = [0, 1, 1, 3, 0]

correct = 0
for pred, label in zip(predicted, labels):
    if pred == label:
        correct += 1

accuracy = correct / len(labels)
print(accuracy)
```

思考：

1. 总共有几个样本？
2. 预测对了几个？
3. 准确率是多少？

19.2 练习 B：保存实验配置

```
from pathlib import Path

config = {
    "model": "SmallCNN",
    "batch_size": 64,
    "learning_rate": 0.001,
    "epochs": 2,
}

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)

config_path = output_dir / "config.txt"
with config_path.open("w", encoding="utf-8") as f:
    for key, value in config.items():
```

```
f.write(f"{key}: {value}\n")

print("配置已保存: ", config_path)
```

19.3 练习 C：画 loss 曲线

```
from pathlib import Path

import matplotlib.pyplot as plt

losses = [0.9, 0.7, 0.5, 0.35, 0.25]
epochs = list(range(1, len(losses) + 1))

output_dir = Path("outputs")
output_dir.mkdir(parents=True, exist_ok=True)

plt.figure()
plt.plot(epochs, losses, marker="o")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.title("Loss Curve")
plt.savefig(output_dir / "practice_loss.png", dpi=150)
plt.close()
```

19.4 练习 D：理解 batch

```
import numpy as np

batch = np.random.randn(4, 1, 28, 28)

print(batch.shape)
print("batch size:", batch.shape[0])
print("channels:", batch.shape[1])
print("height:", batch.shape[2])
print("width:", batch.shape[3])
```

问题：

1. 这个 batch 有几张图片？
2. 每张图片几个通道？
3. 每张图片的高和宽是多少？

19.5 练习 E：写一个简单模型类

```
class SimpleModel:
    def __init__(self, weight, bias):
        self.weight = weight
        self.bias = bias
```

```
def forward(self, x):  
    return self.weight * x + self.bias  
  
model = SimpleModel(weight=2.0, bias=1.0)  
  
for x in [1, 2, 3]:  
    y = model.forward(x)  
    print(f"x={x}, y={y}")
```

这个练习是为了理解后续 PyTorch 里的：

```
output = model(images)
```

模型就是一个把输入变成输出的对象。

20. 学完本手册后要达到的水平

学完后，你不需要成为 Python 高手，但应能做到：

1. 能打开终端并激活课程环境。
2. 能在 VS Code 中选择正确 Python 解释器。
3. 能运行 `.py` 文件。
4. 能看懂变量、列表、字典、循环、函数。
5. 能看懂简单类和 `self`。
6. 能安装 Python 包并理解 `import`。
7. 能使用 `Path` 创建输出目录。
8. 能用 NumPy 理解数组 shape。
9. 能用 Matplotlib 保存曲线或图片。
10. 能理解 tensor、device、batch、argmax 这些后续实验关键词。

只要达到这些要求，就可以进入 MNIST 和 PyTorch 训练实验。

21. 参考资料

1. Python 官方下载页面：<https://www.python.org/downloads/>
2. Miniconda 官方说明：<https://www.anaconda.com/docs/getting-started/miniconda/main>
3. Miniconda 安装指南：<https://www.anaconda.com/docs/getting-started/miniconda/install/overview>
4. VS Code Python 官方说明：<https://code.visualstudio.com/docs/languages/python>
5. VS Code Python 快速开始：<https://code.visualstudio.com/docs/python/python-quick-start>
6. PyTorch 官方安装页面：<https://docs.pytorch.org/get-started/locally/>

